

Development standards and checklists

There are many best practices for development that should be followed by any developer as part of the daily work. Still, under pressure or if there are small adaptations done to an existing component, some of these standards may not be followed.

Looking at experiences e.g. from aviation, checklists can help to ensure that even the simple and standard tasks are not forgotten, increasing the quality of work (and in aviation saving many lives).

The following lists are intended to act as a reminder and should be used as:

- A reminder when coaching junior developers to ensure that best practices are followed
- A reminder for everyone when working on a code not to forget
- A structure for code reviewers and testers

The following lists should be attached to every Java ticket with checkmarks being added by developer, reviewer and tester.

Most important best practices for development

#	Description	BE	FE	checked by Developer	checked by Reviewer	checked by Tester
	General coding standards					
	Code easy to understand and commented	X	X			
	Use Intention-Revealing Names	X	X			
	Do one Thing (in a class or method/function)	X	X			
	Code is well structured	X	X			
	No dead code	X	X			
	Is the code using common libraries in latest version and have libraries been checked for vulnerabilities	X	X			
	Input values checked	X	X			
	No hardcoded values	X	X			
	Resilient against unexpected or missing data	X	X			
	Release resources (Streams, Connections, etc) in all cases	X	X			
	Error handling	X	X			
	Are edge cases and potential error scenarios handled appropriately?	X	X			
	Are error scenarios documented so that testers can verify?	X	X			
	Are all functional errors (errors that are not due to a technical failure like password does not match etc.) mapped to an appropriate error code and message (no http 500) ?	X				
	Are all technical errors mapped to an appropriate error code and message (and not internally mapped to a functional error code)?	X				
	Are errors logged in reporting logs?	X				
	All catch statements either throw an exception or return an error	x	x			
	Database					
	Is any access to database done in a data access layer?	X				
	If database query was added or updated or new fields have been added: Have necessary indexes been added or updated	X				
	Are indexes added or updated by code or have they been added to the pipeline	X				
	Are indexes documented	X				
	Write conflicts handled properly (e.g. when 2 concurrent calls for same data arrive)	X				
	Database synchronization latency issues handled properly	X				
	Locally tested for performance and resource consumption?	X				
	Impact on connection pooling evaluated	X				

	Secret and key handling				
	Do we have a design and implementation to handle expired keys gracefully for the customer?	X			
	No secrets logged anywhere	X			
	Secrets taken from keystore	X			
	Logging				
	Reporting logs for all INBOUND calls	X			
	Reporting logs for all OUTBOUND calls	X			
	Reporting logs have all data	X			
	Reporting logs visible in Dynatrace				
	transaction_id propagated	X			
	KPI logs defined and documented	X			
	KPI logs implemented	X			
	Analytics				
	Tags have been included		X		
	Accessibility				
	Accessibility checked		X		
	Developer tests				
	Unit tests (positive and negative) covering all backend code?	X			
	Are long running / blocking calls handled properly?	X			
	Local test for resource consumption and load executed	X			
	Stubs used during development provided to testers (for load tests)?	X			
	Other (based on issues seen earlier)				
	initialisation or first setup call is completed before any subsequent BE call is triggered	X			